université
PARIS-SACLAY

RAPPORT DE TP

Architecture avancée

Auteur :

Gaëtan BOUCHY
Thomas ABOU JAOUDE
Victor BERCY

Enseignant :

M. HAMMAMI

Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun en trinôme et qu'il a été rédigé de manière autonome.

À Orsay, le 14/11/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
1 - Modélisation et évaluation de mémoires caches 3
1.1 Famille de processeurs ARM 3
1.2 Simulation d'architecture CACTI 3
1.2.1 Analyse de paramètres : le temps d'accès, la taille du cache et l'associativité, la surface et la consommation énergétique 3
1.2.2 Analyse de paramètres : le nombre de ports de lecture et d'écriture 7
1.2.3 Analyse de paramètres : Le mode d'accès au cache 9
1.2.4 Calcul du TAMM 12
1.2.5 Choix des meilleures caractéristiques : TAMM, surface du cache et consommation énergétique 13

université PARIS-SACLAY

1 - Modélisation et évaluation de mémoires caches

1.1 Famille de processeurs ARM

Pour la suite de ce TP, nous avons décidé de nous baser sur le processeur ARM Cortex-A55. D'après la documentation technique, le processeur Cortex-A55 possède un cache L1 Instruction / Data de taille allant de 8 à 64 kB, un cache L2 de taille allant de 64 à 256 kB et enfin un cache L3 optionnel de taille allant de 256 kB à 4 MB. Ainsi, nous allons utiliser ces différents paramètres pour la suite du TP. L'objectif de ce TP est de trouver la configuration optimale afin d'avoir un temps d'accès moyen à la mémoire (TAMM) inférieur 3 ns ainsi qu'une consommation énergétique et une surface de cache minimales.

1.2 Simulation d'architecture CACTI

1.2.1 Analyse de paramètres : le temps d'accès, la taille du cache et l'associativité, la surface et la consommation énergétique

Nous cherchons maintenant à tracer les figures suivantes : le temps d'accès en fonction de la taille du cache et de l'associativité, puis le temps d'accès en fonction de la surface et de l'énergie. Pour chacune de ces figures, nous avons décidé de fixer le nœud technologique soit la demi-distance minimale entre deux motifs lithographiés (deux lignes de métal, par exemple) à 0.032 µm soit 32 nm.

```
# Multiple banks connected using a bus
-UCA bank count 1
-technology (u) 0.032
//-technology (u) 0.045
//-technology (u) 0.068
//-technology (u) 0.090
```

Figure 1 : Choix du nœud technologique

Afin de tracer la première figure qui correspond au temps d'accès en fonction de la taille du cache et de l'associativité, nous devons donc réaliser un script python afin d'automatiser les tailles. Avec le processeur Cortex-A55, nous avons 10 tailles de cache différentes allant de 8 kB jusqu'à 4 MB. Nous avons ainsi un tableau sizes contenant ces 10 tailles dans notre code :

```
sizes = [8192, 16384, 32768, 65536, 131072, 262144, 524288, 1048576, 2097152, 4194304]
```

Figure 2 : Variable « sizes »

Puis nous avons également 4 niveaux d'associativité différents allant de 1 à 8. Nous avons ainsi un tableau associativité contenant ces 4 valeurs dans notre code :

```
associativity = [1, 2, 4, 8]
```

Figure 3 : Variable « associativity »

Notre objectif est donc d'extraire le temps d'accès pour chaque taille ainsi que chaque niveau d'associativité. Nous aurons ainsi 40 valeurs (10 tailles de caches * 4 niveaux d'associativités).

Nous avons donc réalisé un script Python qui va lancer la commande « ./cacti - infile cache.cfg » puis modifier le fichier « .cfg » afin d'y changer la taille du cache ou le niveau d'associativité.

```
def write_cfg(s, a):
        with open("cache.cfg", "r") as f:
                txt = f.readlines()
        f.close()
        with open("cache.cfg", "w") as f:
                for line in txt:
                        if "-size (bytes)" in line:
                                line = "-size (bytes) " + f"{s}\n"
                        elif "-associativity" in line:
                                line = "-associativity " + f"{a}\n"
                        f.write(line)
        f.close()
```

Figure 4 : Code permettant de changer la taille du cache ou le niveau d'associativité du fichier

Puis nous réalisons deux boucles for, la première de la taille du tableau « sizes » et la deuxième de la taille du tableau « associativity » où nous appelons la fonction write_cfg(). Après avoir récupérer toutes les valeurs des temps d'accès, nous réalisons notre graphe en 3D.

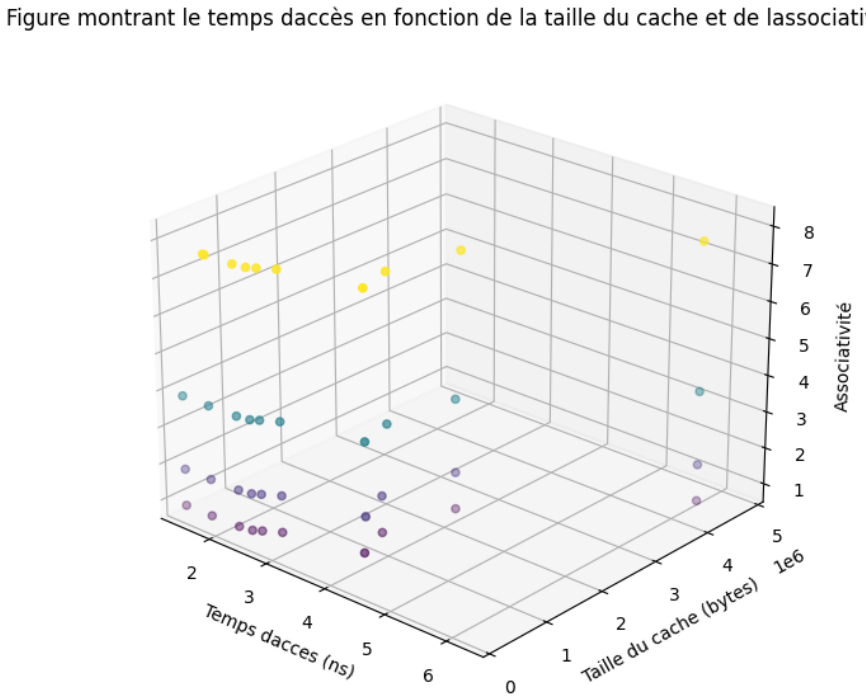


Figure 5 : Figure montrant le temps d'accès en fonction de la taille du cache et de l'associativité

Comme nous pouvons le constater sur ce graphe en trois dimensions, la taille du cache joue un impact important sur le temps d'accès. En effet, il semblerait que plus la taille du cache soit importante, plus le temps d'accès l'est également.

D'un autre côté, il semblerait que l'associativité impacte également le temps d'accès mais dans une moindre mesure. Cependant, cela est particulièrement flagrant lorsque la taille du cache est faible. En effet, lorsque l'associativité est à son maximum soit 8, le temps d'accès augmente comme le montre la forme ovale rouge.

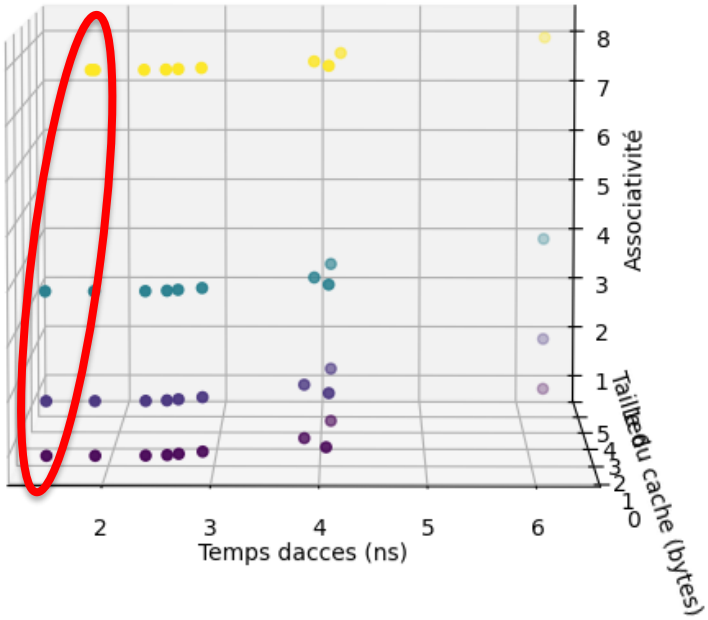


Figure 6 : Figure montrant l'impact de l'associativité sur le temps d'accès

D'un autre côté, l'associativité joue un rôle important sur le calcul du temps d'accès moyen à la mémoire (TAMM) comme le montre la formule suivante :

$$TAMM = t_{aL1} + \tau_{eL1} \times (t_{aL2} + \tau_{eL2} \times t_M)$$

Figure 7 : Formule de calcul du TAMM

En effet, plus l'associativité est élevée, plus le taux d'échec (τ_{eL}) diminue. Ainsi, nous avons estimé que lorsque notre associativité valait 8, notre taux d'échec serait de 0.02, avec une associativité de 4, il serait de 0.04, avec une associativité de 2, il serait de 0.06 et enfin, avec une associativité de 1, il serait de 0.01.

Afin de tracer la deuxième figure qui correspond au temps d'accès en fonction de la surface et de la consommation énergétique, nous prenons le même modèle que nous avons vu précédemment. Nous avons donc là encore réalisé un script Python qui va lancer la commande « ./cacti - infile cache.cfg » puis modifier le fichier « .cfg » afin d'y changer la taille du cache ou le niveau d'associativité. Nous récupérons donc les données du temps d'accès, de la surface du cache et de la consommation énergétique :

Figure montrant le temps daccès en fonction de la surface du cache et de la consommation énergétique

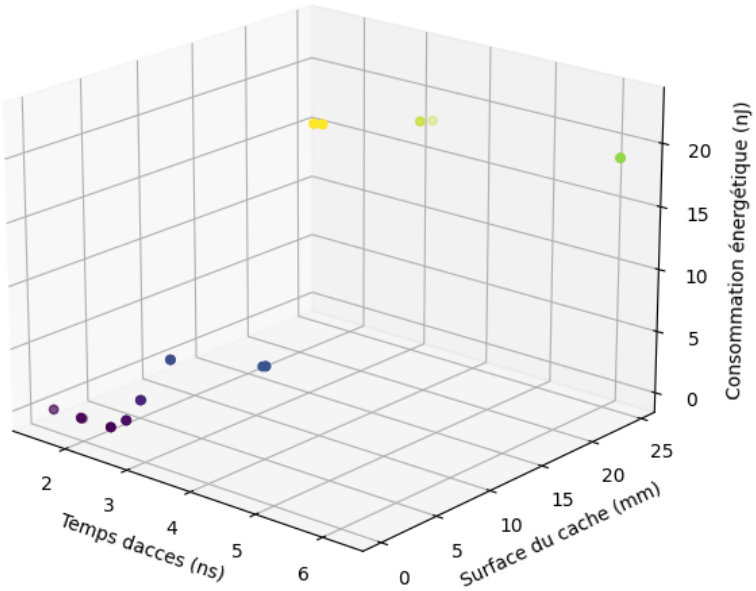


Figure 8 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique

Nous pouvons voir sur ce graphe en trois dimensions que plus la surface du cache est importante, plus le temps d'accès l'est également. De plus, la consommation énergétique l'est aussi. On peut donc voir une augmentation de ces trois caractéristiques de manière simultanée.

En effet, une augmentation de la surface du cache entraîne une augmentation du temps d'accès ainsi que de la consommation énergétique.

1.2.2 Analyse de paramètres : le nombre de ports de lecture et d'écriture

Nous avons donc là encore modifié notre script Python qui va lancer la commande « ./cacti - infile cache.cfg », puis modifier le fichier « .cfg » afin d'y changer le nombre de ports de lecture et d'écriture.

```
def write_cfgPort(readWrite, read, write):
        with open("cache.cfg", "r") as f:
                txt = f.readlines()
        f.close()
        with open("cache.cfg", "w") as f:
                for line in txt:
                        if "-exclusive read port" in line:
                                line = "-exclusive read port " + f"{read}\n"
                        elif "-exclusive write port" in line:
                                line = "-exclusive write port " + f"{write}\n"
                        elif "-read-write port" in line:
                                line = "-read-write port " + f"{readWrite}\n"
                        f.write(line)
        f.close()
```

Figure 9 : Code permettant de changer le nombre de ports de lecture et d'écriture du fichier

Nous avons testé cela pour huit cas différents où nous avons soit donné une valeur au port read-write, soit donné une valeur au port de lecture et au port d'écriture.

```
ports = [(1,0,0),(0,1,1),(3,0,0),(0,3,3),(5,0,0),(0,5,5),(10,0,0),(0,10,10)]
```

Figure 10 : Test des différents ports de lecture et d'écriture

Nous récupérons donc les données du temps d'accès, de la surface du cache et de la consommation énergétique pour différents modes de lecture et d'écriture qui sont affichés sur le graphe ci-dessous :

Figure montrant le temps daccès en fonction de la surface du cache et de la consommation énergétique

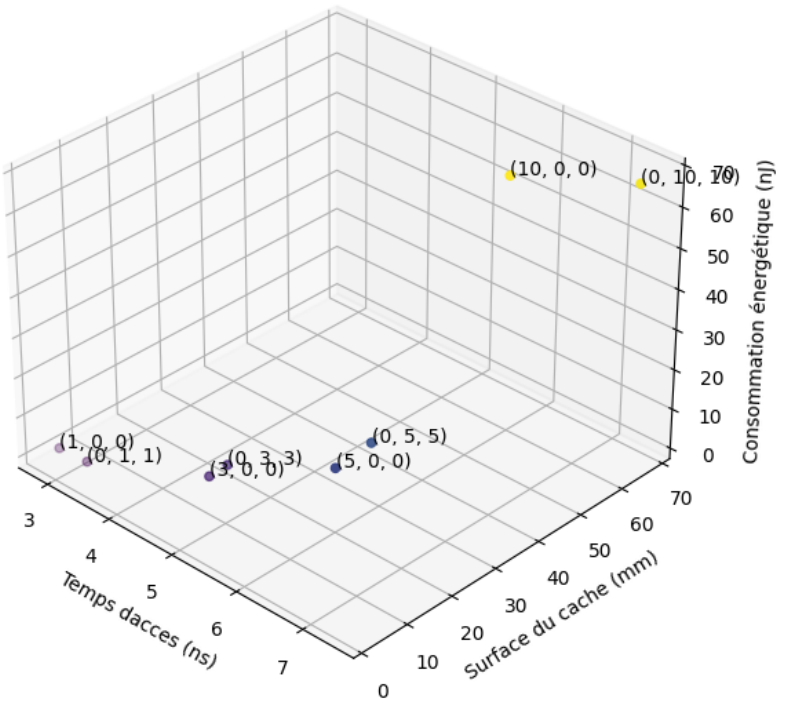


Figure 11 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique pour différents modes de lecture et d'écriture

Nous pouvons voir que plus le nombre de « read-write port » ou que le nombre de « exclusive read port » et de « exclusive write port », plus le temps d'accès augmente ainsi que la surface du cache et la consommation énergétique.

D'un autre point de vue, nous obtenons également ce graphe qui montre davantage l'impact du nombre de ports sur le temps d'accès, la surface du cache et la consommation énergétique :

Figure montrant le temps daccès en fonction de la surface du cache et de la consommation énergétique

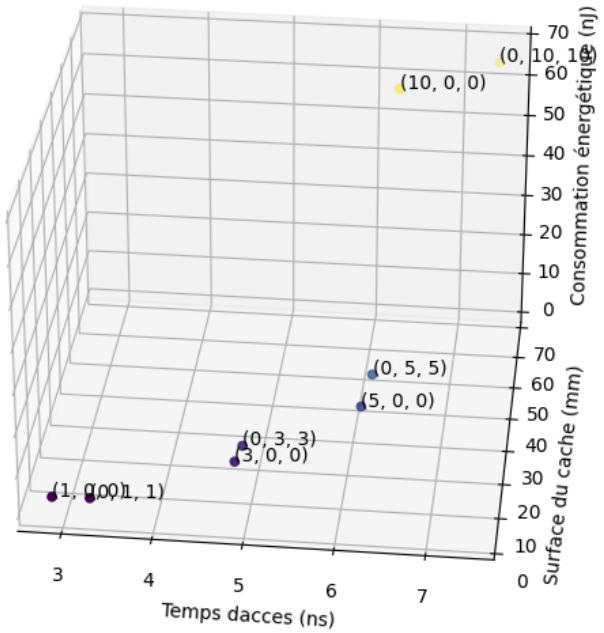


Figure 12 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique pour différents modes de lecture et d'écriture

Ainsi, pour la suite, nous avons choisi de garder le mode « read-write port » avec comme valeur « 1 » car il semblerait qu'avec ce mode, nous ayons les meilleures performances en termes de temps d'accès, de surface du cache et de consommation énergétique.

1.2.3 Analyse de paramètres : Le mode d'accès au cache

Nous avons donc là encore modifié notre script Python qui va lancer la commande « ./cacti - infile cache.cfg », puis modifier le fichier « .cfg » afin d'y le mode d'accès au cache :

```
def write_cfgMode(word):
        with open("cache.cfg", "r") as f:
                txt = f.readlines()
        f.close()
        with open("cache.cfg", "w") as f:
                for line in txt:
                        if "-access mode (normal, sequential, fast)" in line:
                                line = '-access mode (normal, sequential, fast) - "' + word + '"\n'
                        f.write(line)
        f.close()
```

Figure 13 : Code permettant de changer le mode d'accès au cache du fichier

Ainsi, nous avons testé les différents modes d'accès au cache (normal, sequential et fast) pour différents niveaux d'associativité allant de 1 à 8. Voici ce que nous obtenons :

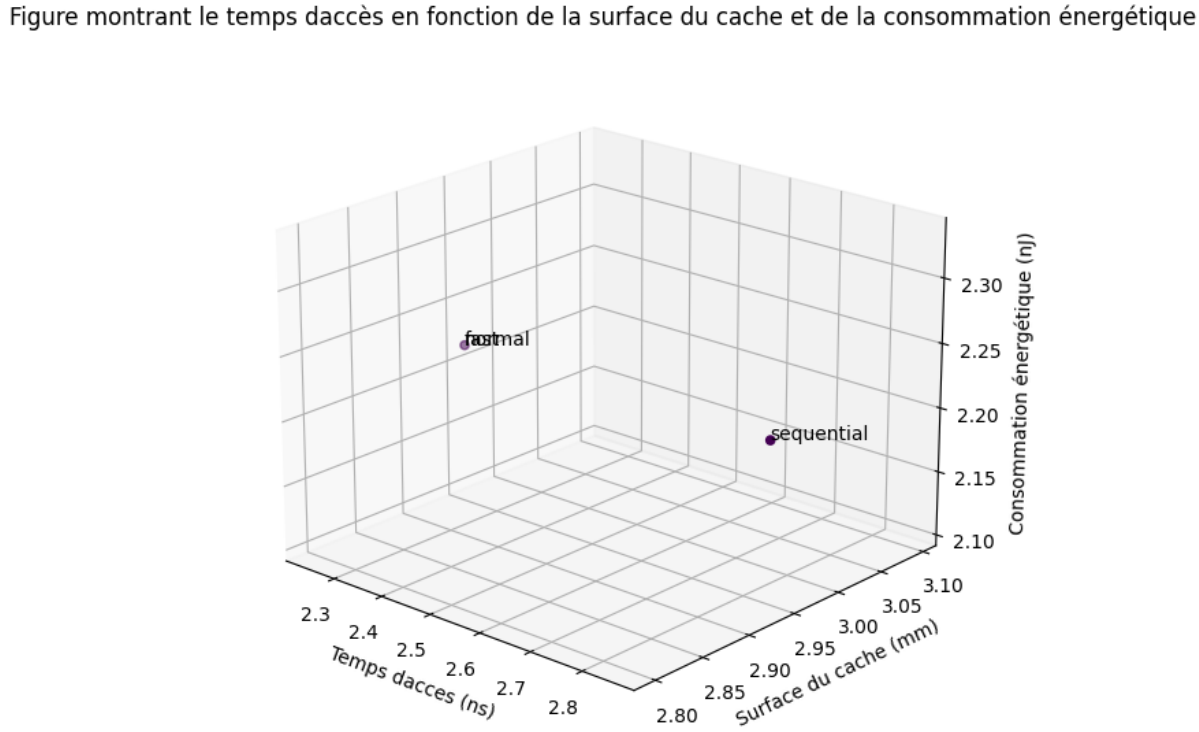


Figure 14 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique pour différents modes d'accès au cache pour une associativité de « 1 »

Figure montrant le temps daccès en fonction de la surface du cache et de la consommation énergétique

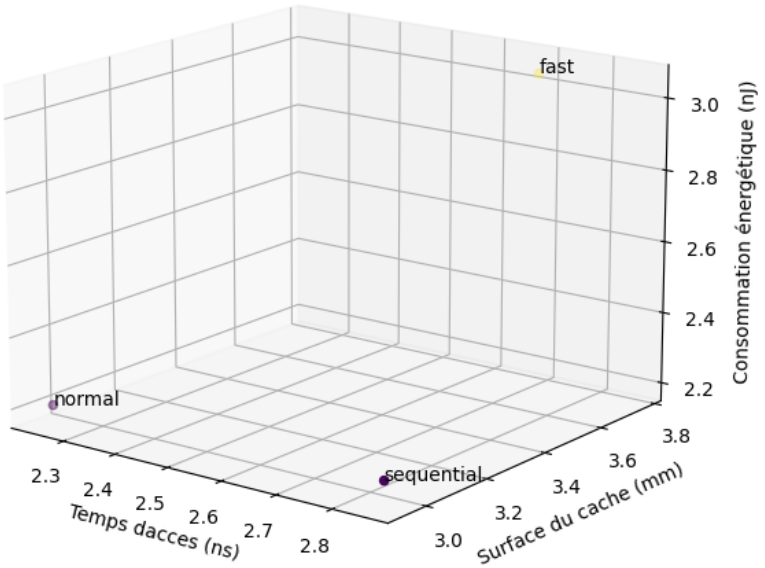


Figure 15 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique pour différents modes d'accès au cache pour une associativité de « 2 »

Figure montrant le temps daccès en fonction de la surface du cache et de la consommation énergétique

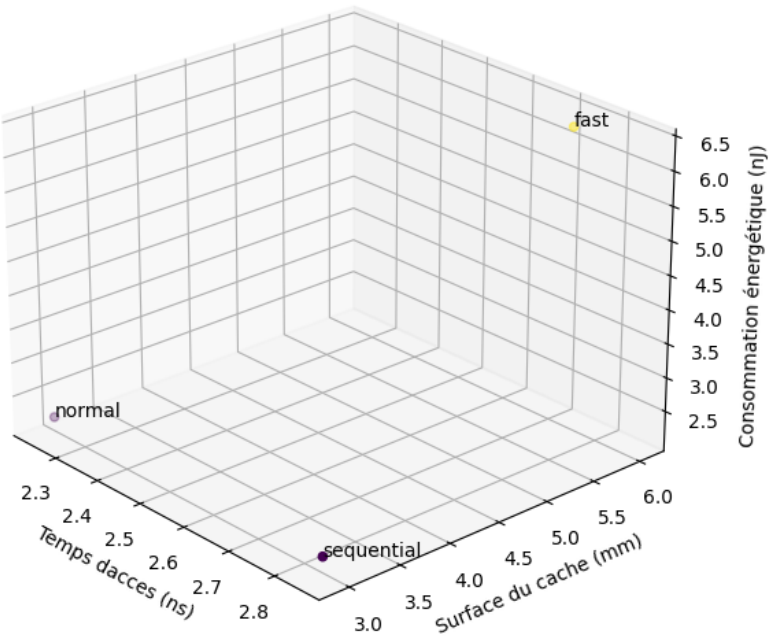


Figure 16 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique pour différents modes d'accès au cache pour une associativité de « 4 »

Figure montrant le temps daccès en fonction de la surface du cache et de la consommation énergétique

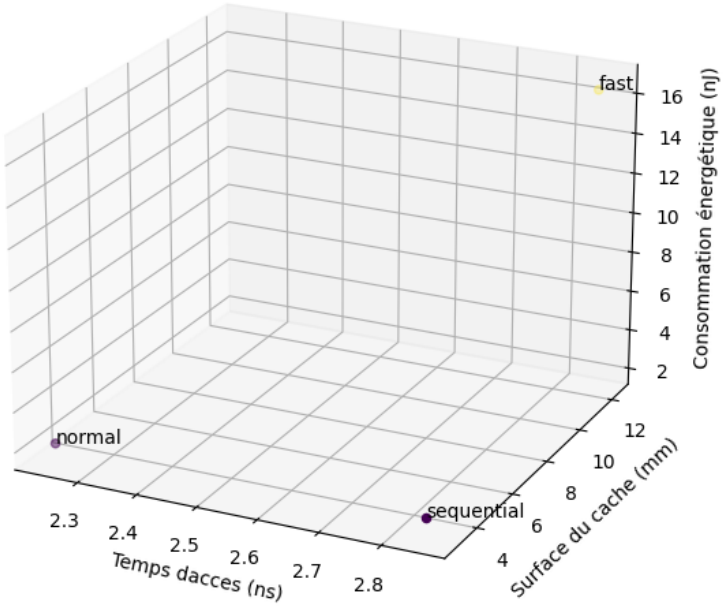


Figure 17 : Figure montrant le temps d'accès en fonction de la surface du cache et de la consommation énergétique pour différents modes d'accès au cache pour une associativité de « 8 »

A COMMENTER

1.2.4 Calcul du TAMM

```
gaetan@gaetan-VirtualBox:~/Bureau/cacti$ python3 python.py
Somme de tout :  2.612712819438
temponL1 : 8192 temponL2 : 65536
temponL3 : 262144
TAMM :  1.36011268 ns
Energie :  0.91999939 nJ
Surface :  0.332600749438 mm
```

1.2.5 Choix des meilleures caractéristiques : TAMM, surface du cache et consommation énergétique